COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	July 24, 2026	**DATE SUBMITTED:**	July 29, 2026
NAME:	Libo-on, Gian Carlo G.	**INSTRUCTOR:**	Engr, Jimlord Quejado

ACTIVITY NO. 1 **Algorithm, Flowchart and Pseudocode**

1. PROCEDURE OUTPUT

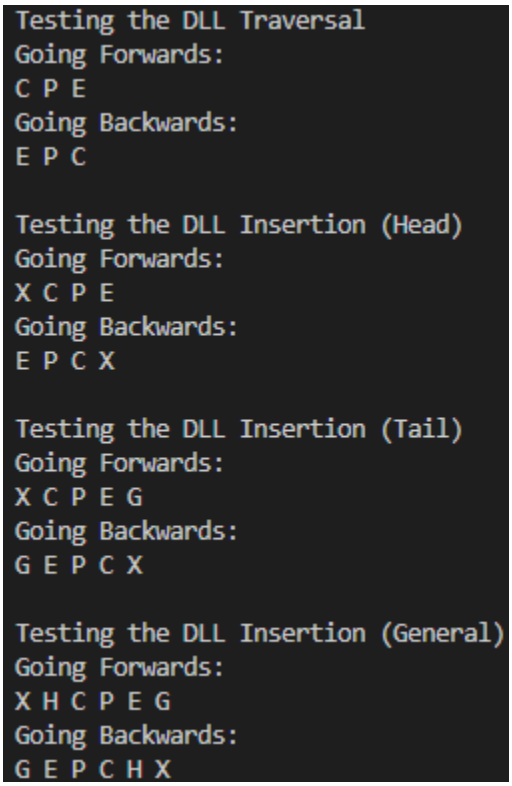
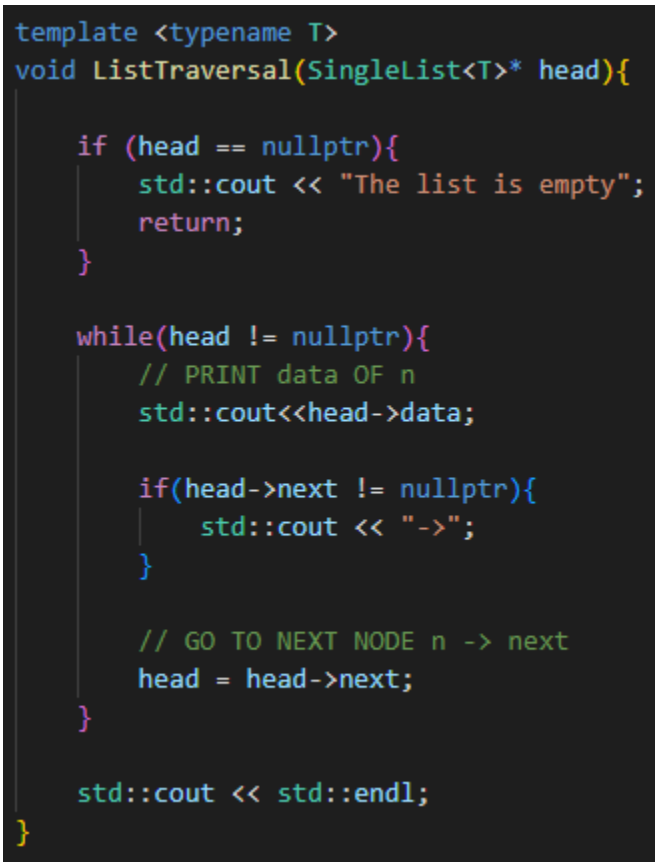
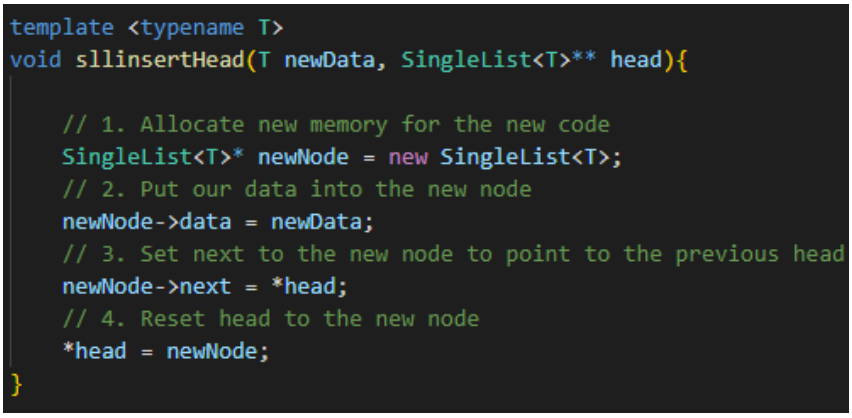
Screenshot	
Discussion	This initial simple implementation lacks the print function since it only displays the elements and not really printing it.

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	
Insertion at head	

```
template <typename T>
void ListTraversal(SingleList<T>* head){

    if (head == nullptr){
        std::cout << "The list is empty";
        return;
    }

    while(head != nullptr){
        // PRINT data OF n
        std::cout<<head->data;

        if(head->next != nullptr){
            std::cout << "->";
        }

        // GO TO NEXT NODE n -> next
        head = head->next;
    }

    std::cout << std::endl;
}
```

```
template <typename T>
void sllinsertHead(T newData, SingleList<T>** head){

    // 1. Allocate new memory for the new code
    SingleList<T>* newNode = new SingleList<T>;
    // 2. Put our data into the new node
    newNode->data = newData;
    // 3. Set next to the new node to point to the previous head
    newNode->next = *head;
    // 4. Reset head to the new node
    *head = newNode;
}
```

Insertion at any part of the list

```
template <typename T>
void sllGeneralInsert(T newData, SingleList<T>* prevNode){

    //1. Check if it is the head node (previous node is null)
    if(prevNode == nullptr){
        //2. If null, print "Previous node cannot be null."
        std::cout<<"Previous value cannot be null"<<std::endl;
        return;
    }
    //3. Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;
    //4. Store data in the new node
    newNode->data = newData;
    //5. Point new node to the node previous node was pointing to
    newNode->next = prevNode->next;
    //6. Point previous node to the new node
    prevNode->next = newNode;
}
```

Insertion at the end

```
template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head){
    // 1. Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;

    // 3. Store data in new node
    newNode->data = newData;
    // 4. Point to null
    newNode->next = nullptr;

    // 2. Dereference the head node
    SingleList<T>* currentNode = *head;

    // 5. Traverse the list
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }

    // 6. Point the next of the current node to the new node
    currentNode->next = newNode;
}
```

Deletion of a node	

```cpp
template <typename T>
void sllDeleteNode(T findData, SingleList<T>** head){

    // if there is nothing to delete
    if (*head == nullptr) return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

    // searching the node: 1. the node does not exist + we found data
    while (currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;

    }

    // if data not found currNode == nullPtr in the while loop
    if(currNode == nullptr){
        std::cout << "the data was not found\n" << std::endl;
        return;
    }

    // if data was found
    if(prevNode == nullptr){
        *head = currNode->next;
    } else {
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

Table 3-2.Code for the List Operations

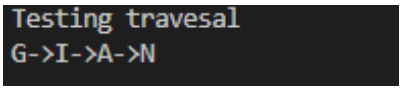
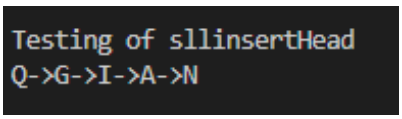
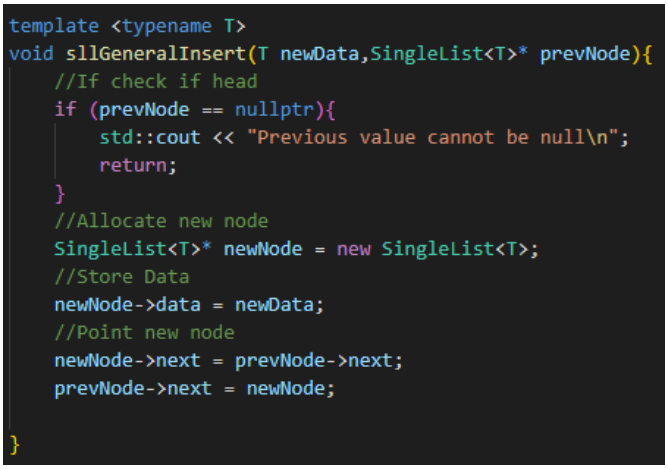
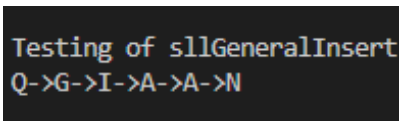
a	**Source Code**	

```cpp
template<typename T>
void ListTravesal(SingleList<T>* head){
    if (head == nullptr){
        std::cout << "There is no list\n";
        return;
    }

    while(head != nullptr){
        std::cout << head->data;

        if (head->next != nullptr) {
            std::cout << "->";
        }

        head = head->next;
    }
    std::cout << std::endl;
}
```

	Console	
b	**Source Code**	
	Console	
c	**Source Code**	
	Console	
d	**Source Code**	

```
template<typename T>
void sllinsertHead(T newData,SingleList<T>** currentHead){
    SingleList<T>* newNode = new SingleList<T>;

    newNode->data = newData;
    newNode->next = *currentHead;
    *currentHead = newNode;

}
```

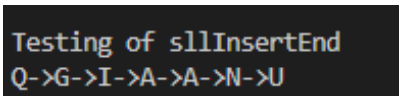
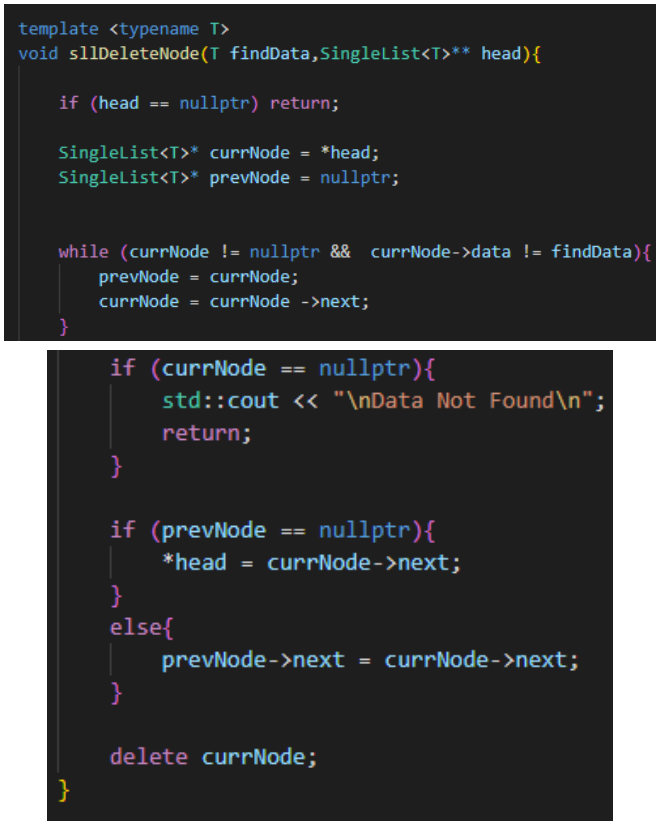
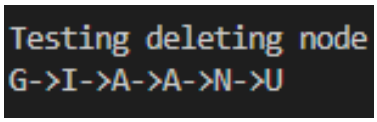
```
template <typename T>
void sllGeneralInsert(T newData,SingleList<T>* prevNode){
    //If check if head
    if (prevNode == nullptr){
        std::cout << "Previous value cannot be null\n";
        return;
    }
    //Allocate new node
    SingleList<T>* newNode = new SingleList<T>;
    //Store Data
    newNode->data = newData;
    //Point new node
    newNode->next = prevNode->next;
    prevNode->next = newNode;

}
```

```
template<typename T>
void sllInsertEnd(T newData, SingleList<T>** head){
    SingleList<T>* newNode = new SingleList<T>;
    SingleList<T>* currentNode = *head;
    newNode->data = newData;
    newNode->next = nullptr;

    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }

    currentNode->next = newNode;
}
```

	Console	
e	**Source Code**	
	Console	

```
template <typename T>
void sllDeleteNode(T findData,SingleList<T>** head){

    if (head == nullptr) return;

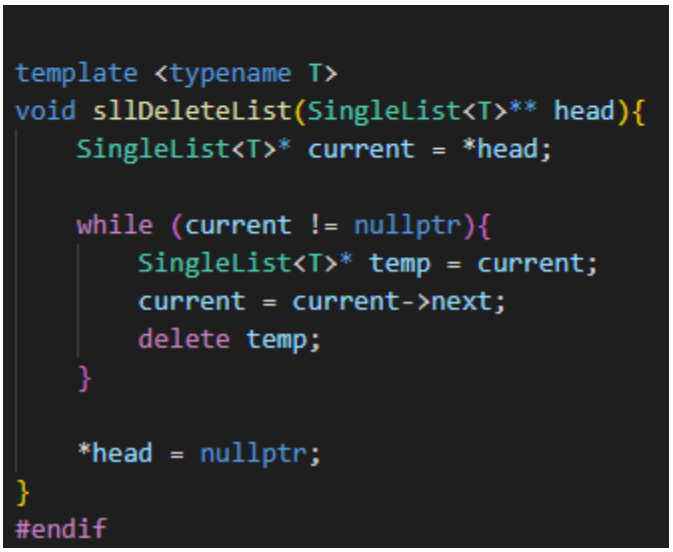
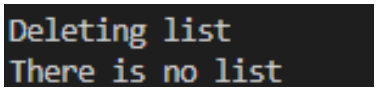
    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;


    while (currNode != nullptr &&  currNode->data != findData){
        prevNode = currNode;
        currNode = currNode ->next;
    }
```

```
    if (currNode == nullptr){
        std::cout << "\nData Not Found\n";
        return;
    }

    if (prevNode == nullptr){
        *head = currNode->next;
    }
    else{
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

f	**Source Code**	
	Console	

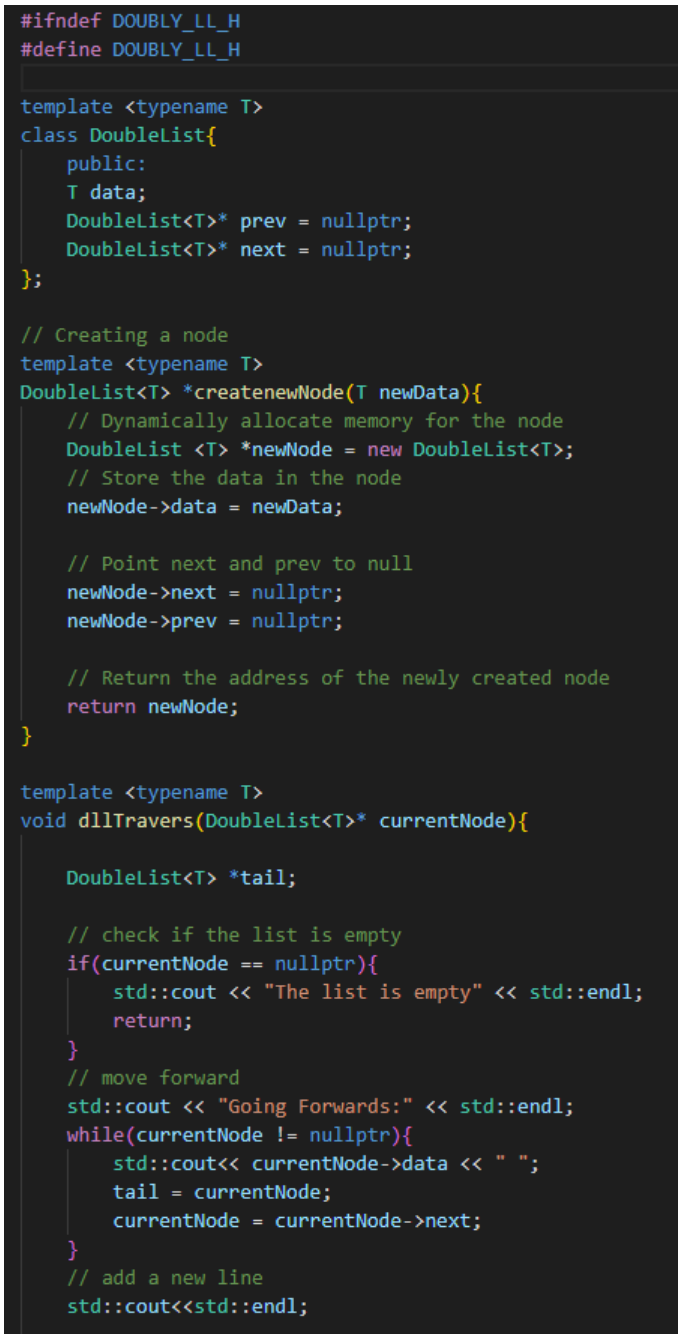
```
template <typename T>
void sllDeleteList(SingleList<T>** head){
    SingleList<T>* current = *head;

    while (current != nullptr){
        SingleList<T>* temp = current;
        current = current->next;
        delete temp;
    }

    *head = nullptr;
}
#endif
```

```
Deleting list
There is no list
```

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshots	Analysis
	This image shows the traverse function which you can go bi-directionally by pointing one to the next, and pointing another to the previous. This allows the program to read both forward and backward elements.

```cpp
#ifndef DOUBLY_LL_H
#define DOUBLY_LL_H

template <typename T>
class DoubleList{
    public:
    T data;
    DoubleList<T>* prev = nullptr;
    DoubleList<T>* next = nullptr;
};

// Creating a node
template <typename T>
DoubleList<T> *createnewNode(T newData){
    // Dynamically allocate memory for the node
    DoubleList <T> *newNode = new DoubleList<T>;
    // Store the data in the node
    newNode->data = newData;

    // Point next and prev to null
    newNode->next = nullptr;
    newNode->prev = nullptr;

    // Return the address of the newly created node
    return newNode;
}

template <typename T>
void dllTravers(DoubleList<T>* currentNode){

    DoubleList<T> *tail;

    // check if the list is empty
    if(currentNode == nullptr){
        std::cout << "The list is empty" << std::endl;
        return;
    }
    // move forward
    std::cout << "Going Forwards:" << std::endl;
    while(currentNode != nullptr){
        std::cout<< currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }
    // add a new line
    std::cout<<std::endl;
```

```
    // move backward
    std::cout << "Going Backwards:" << std::endl;

    while(tail != nullptr){
        std::cout<< tail->data << " ";
        tail = tail->prev;
    }
};

// Insertion at the head
template <typename T>
void dllInsertHead(T newData, DoubleList<T> ** currentHead){
    // creating a new node
    DoubleList<T>* newNode = createnewNode(newData);

    //new node should point to the current head
    newNode->next = *currentHead;

    //current head should point back to the newNode
    (*currentHead)->prev = newNode;

    //update the pointer head
    *currentHead = newNode;
}

template<typename T>
void dllInsertEnd(T newData, DoubleList<T>* currentHead){
    //Create a new Node
    DoubleList<T>* newNode = createnewNode(newData);

    //Traverse until the end of the node
    while(currentHead->next != nullptr){
        currentHead = currentHead->next;
    }
    // connect the end node to the new node
    currentHead->next = newNode;
    // connect the new node back to the last one
    newNode->prev = currentHead;
}
```

This basically just allows you to insert a node both in front and back of the list. After that, you can easily insert new data to that node allowing you to add letters or elements in front and back of your elements.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```h
#ifndef S_DOUBLE_LL_H
#define S_DOUBLE_LL_H
#include <iostream>
#include <string>

//blueprint for the song
template <typename T>
class music_playlist{
    public:
        T data;
        std::string artist;
        music_playlist<T>* next; // for storing the next song
        music_playlist<T>* prev; // for storing the prev song
};

//Creating nodes
template <typename T>
music_playlist<T>* new_Node(T newData, std::string newArtist){
    //Dynamic allocation for a new node
    music_playlist<T>* newNode = new music_playlist<T>;
    //store the music in a new node
    newNode->data = newData;
    newNode->artist = newArtist;
    //Point it to null after
    newNode->next = nullptr;
    newNode->prev = nullptr;
    return newNode;
}
```

Image 1. Templating H file

This image shows how the general template of the objects used in the main function is created. As you can see, there's a blueprint on the format of the song. On the lower part, shows the creation of nodes in the playlist.

```h
//MUSIC INSERTION
template <typename T>
void addmusic(T newData, std::string newArtist, music_playlist<T>** currenthead){
    //create a new node
    music_playlist<T>* newNode = new_Node(newData, newArtist);
    //If playlist is empty
    if(*currenthead == nullptr){
        //the first song points back to itself
        newNode->next = newNode;
        newNode->prev = newNode;
        *currenthead = newNode;
        return;
    }
    //Get the last node
    music_playlist<T>* tail = (*currenthead)->prev;
    //connect last node to a new node
    newNode->next = *currenthead;
    newNode->prev = tail;
    tail->next = newNode;
    (*currenthead)->prev = newNode;

};
```

Image 2. Music Insertion Function

This image shows how music is added in the playlist. If there is no song, the first inserted song will point back to itself. When you continue adding songs, then more nodes will be added and the previous last node will connect to that new node.

```cpp
//Next song
template <typename T>
void next_music(music_playlist<T>*& currentnode){
    if (currentnode == nullptr){
        std::cout << "Playlist is empty. \n";
        return;
    }
    currentnode = currentnode->next;
    std::cout << "Currently Playing: " << currentnode->data << " by " << currentnode->artist << "\n";
};

// Previous
template <typename T>
void prev_music(music_playlist<T>*& currentnode){
    if (currentnode == nullptr){
        std::cout << "Playlist is empty. \n";
        return;
    }
    currentnode = currentnode->prev;
    std::cout << "Currently Playing: " << currentnode->data << " by " << currentnode->artist << "\n";
};

// View all songs
template <typename T>
void view_all_music(music_playlist<T>* currentnode){
    if (currentnode == nullptr){
        std::cout << "Playlist is empty. \n";
        return;
    }
    music_playlist<T>* start = currentnode;
    int song = 1;
    do {
        std::cout << song << "." << currentnode->data << " by " << currentnode->artist << "\n";
        song++;
        currentnode = currentnode->next;
    } while (currentnode != start);
};
```

Image 3. Next, Previous, and View Functions

This is basically just a traversing function. Next allows the user to move to the next song by pointing the current node to the next node to move. The previous is just the same function but pointing backward. There is also the view songs which basically shows every song that you have entered in the playlist along with their artist.

```cpp
template <typename T>
void Del_music(int pos, music_playlist<T>** currenthead){
    if (*currenthead == nullptr){
        std::cout << "Playlist is empty.";
        return;
    }

    if (pos < 1) {
        std::cout << "Invalid position.\n";
        return;
    }

    music_playlist<T>* currentnode = *currenthead;
    //move to the selected position
    for(int i = 1; i < pos; i++){
        currentnode = currentnode->next;
        if (currentnode == *currenthead) {
            std::cout << "Position out of bounds.\n";
            return;
        }
    }

    if(currentnode->next == currentnode){
        *currenthead = nullptr;
        delete currentnode;
        return;
    }

    currentnode->prev->next = currentnode->next;
    currentnode->next->prev = currentnode->prev;

    if (currentnode == *currenthead){
        *currenthead = currentnode->next;
    }
    delete currentnode;
};

#endif
```

Image 4. Delete functions

This image shows the deletion function of the H file. The if case for position allows for error management which could happen if the position is less than 1. The for loop allows the user to move through all of the nodes and find what song you want to delete.

```cpp
#include <iostream>
#include <string>
#include <limits>
#include "Sample_ll.h"

int main (){

    music_playlist<std::string>* head = nullptr;
    int choice;

            do {
                std::cout << "\n===== MUSIC PLAYLIST =====\n";
                std::cout << "1. Add Song\n";
                std::cout << "2. View All Songs\n";
                std::cout << "3. Next Song\n";
                std::cout << "4. Previous Song\n";
                std::cout << "5. Remove Song\n";
                std::cout << "6. Exit\n";
                std::cout << "Enter choice: ";
                std::cin >> choice;

                switch (choice) {
```

Image 5. Header file and interface

This basically just shows the header file and the interface for the TUI which prints out the possible choices for the user to choose from.

```cpp
case 1:
{
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    std::string song, artist;
    std::cout << "Enter new music: ";
    std::getline(std::cin, song);
    std::cout << "Enter artist name: ";
    std::getline(std::cin, artist);
    addmusic<std::string>(song, artist, &head);
    break;
}

case 2:
{
    std::cout << "\nPlaylist:\n";
    view_all_music(head);
    break;
}

case 3:
{
    next_music(head);
    break;
}

case 4:
{
    prev_music(head);
    break;
}

case 5:
{
    int number;
    std::cout << "Enter a song number to remove: ";
    std::cin >> number;
    Del_music(number, &head);
    break;
}

case 6:
{
    std::cout << "Exiting Playlist.\n";
    break;
}
```

```cpp
                default:
                {
                    std::cout << "Invalid choice.\n";
                }
            }
        } while (choice != 6);
    return 0;
}
```

Image 6. Switch case

This basically just shows all the cases for the switch case which you can use to navigate and call the functions that you have declared in the H file.

```
===== MUSIC PLAYLIST =====
1. Add Song
2. View All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 1
Enter new music: I Love You So
Enter artist name: The Walters

===== MUSIC PLAYLIST =====
1. Add Song
2. View All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 1
Enter new music: Atlantis
Enter artist name:  Seafret
```

Image 7. Output for Adding Music

```
===== MUSIC PLAYLIST =====
1. Add Song
2. View All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 2

Playlist:
1.I Love You So by The Walters
2.Atlantis  by  Seafret
3.Yellow by Coldplay
4.Mrs. Potato Head by Melanie Martinez
5.i still talk to jesus by Lany
6.anything 4 u by Lany
7.You'll Be Safe Here by Adie
8.Sparks by Coldplay
9.Fireflies by Owlcity
10.i swear i'll never leave again by keshi
11.lose by NIKI
12.IRIS by The Goo Goo Dolls
13.Favorite Crime by Olivia Rodrigo
14.Until I found you by Stephen Sanchez
15.Thru These Tears by Lany
16.Always by keshi
17.Enchanted by Taylor Swift
18.I Don't Wanna Be Okay Without You by Charlie Burg
19.Those Eyes by New West
20.Pluto Projector by Rex Orange County
21.Strawberries & Cigarettes by Troye Sivan
22.This Side Of Paradise by Coyote Theory
23.An Art Gallery Could Never Be As Unique As You by mrld
24.Be my mistake by The 1975
```

Image 8. Output for View All Songs



Image 9. Output for Next and Previous functions

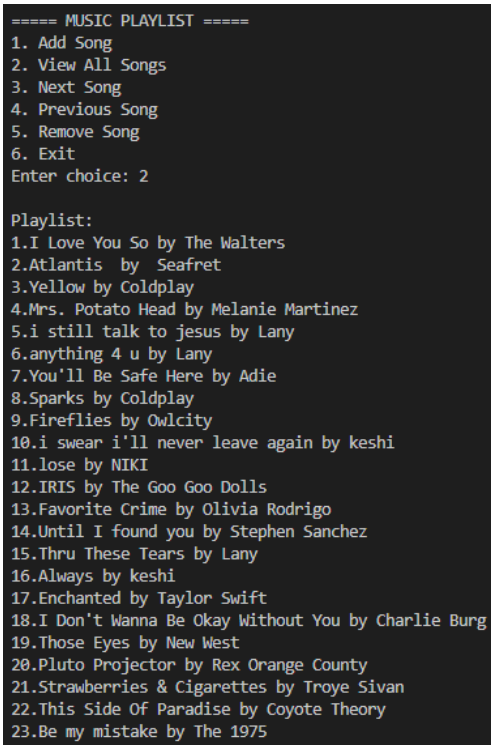


Image 10. Output for Deleting song

Conclusion:

Linked Lists are data structures that allow dynamic memory allocation and better insertion and deletion of data. Single link lists allow one directional movement which makes it better for memory management. On the other hand, Double link lists allow two directional movement which makes it better for data processing and speed. It allows you to traverse the data without relying on the head unlike the single link list. However, it has double the memory of a single linked list.

After this activity, I think that learning about linked lists allows me to understand data management better. It helps me handle the data much better and more efficiently. In the supplementary activity, I made use of linked lists in order to store data, traverse through the data set, delete a part of the data, etc. By doing this I came to understand better how to navigate and implement linked lists in programming and data management.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.